Cafenea — din Java vanilla în Spring

Alex · cerințe pe etape

Pornire

- Proiectul vanilla e în `cafenea-vanilla-java/`. Rulează `app.Main` și încearcă meniul.
- Creează proiectul Spring pe start.spring.io: Maven · Java 21 · Spring Boot 3.5.x · Group `app` · Artifact `cafenea` · fără dependențe. Dezarhivează-l în `cafenea-spring/`, lângă `cafenea-vanilla-java/`.
- Copiază pachetele `produse/`, `comenzi/` și `view/` în pachetul `app` al proiectului Spring.
- Fiecare etapă pornește de la cea dinainte. La final de etapă: un commit `etapa N`, pușat. `cafenea-vanilla-java/` rămâne neschimbat.
- La fiecare etapă folosești doar adnotările ei și pe cele din etapele anterioare. Mesajele dintre `[ ]` se afișează exact ca în cerință.
- Argumentele de forma `--cheie=valoare` se pun în IntelliJ la Run → Edit Configurations → Program arguments.

Etapa 1 · Componente și bean-uri

Adnotări noi: `@SpringBootApplication`, `@ComponentScan`, `@Component`, `@Configuration`, `@Bean`, `@Import`

Cerințe

- Clasele `*Singleton` se șterg. În proiect nu mai există niciun `getINSTANCE()`.
- Clasele din `repository/`, `service/` și `view/` nu-și mai creează singure dependențele (nici `getINSTANCE()`, nici `new`): le primesc prin constructor.
- `Main` dispare. Meniul pornește dintr-un `CommandLineRunner`, în clasa `CafeneaApplication`.
- `Scanner` și `CalculatorPret` (TVA 0.21) devin bean-uri în `AppConfig`.
- `CafeneaApplication` stă în pachetul `app.pornire`, iar `AppConfig` în pachetul `config` (în afara lui `app`). Aplicația trebuie să pornească așa.

Verificare

Opțiunile `2, 2, 2, 4, 3, 4, 5, 0` dau exact aceeași ieșire ca în proiectul vanilla:

```
Bon comanda #1
  Cappuccino    12.50
  Croissant      9.50
  Subtotal      22.00
  TVA            4.62
  TOTAL         26.62 lei
```

La opțiunea 5: `#1  2 produse   26.62 lei`

Etapa 2 · Roluri

Adnotări noi: `@Service`, `@Repository`

Cerințe

- Clasele `*Impl` din `service/` primesc `@Service`, cele din `repository/` primesc `@Repository`, în locul lui `@Component`.

Verificare

Ieșirea e identică cu etapa 1.

Etapa 3 · Configurare din fișier

Adnotări noi: `@Value`, `@ConfigurationProperties`, `@EnableConfigurationProperties`, `@PropertySource`

Cerințe

- `application.yml`, sub cheia `cafenea`: `nume: Cafenea`, `moneda: lei`, `tva: 0.21`. Boot îl încarcă singur.

- `program.properties` (în `resources/`): `cafenea.program=08:00-20:00`, încărcat cu `@PropertySource` pe `AppConfig`. Rămâne `.properties`: `@PropertySource` nu citește YAML.
- `CalculatorPret` primește TVA-ul cu `@Value` și devine `@Component`. În `AppConfig` rămâne un singur `@Bean`: `Scanner`.
- `record CafeneaProperties(String nume, String moneda, String program)` în pachetul `config`, cu `@ConfigurationProperties(prefix = "cafenea")`, activat cu `@EnableConfigurationProperties` pe `AppConfig`.
- Antetul meniului devine `=== <nume> · <program> ===`. Textul `lei` din `ViewCafenea` vine din `moneda`.

Verificare

Antetul: `=== Cafenea · 08:00-20:00 ===`. Cu `--cafenea.tva=0.19`, bonul din etapa 1 are `TOTAL 26.18 lei`.

Etapa 4 · Mai multe bean-uri de același tip

Adnotări noi: `@Autowired`, `@Primary`, `@Qualifier`, `@Order`

Cerințe

- Pachetul `comenzi/bon/`: interfața `BonPrinter` cu `void printeaza(Comanda comanda)`.
- `SimpluBonPrinter`, bean cu numele `simplu`, implicit (`@Primary`): produsele și `TOTAL`. Îl folosește opțiunea 3.
- `DetaliatBonPrinter`, bean cu numele `detaliat`: produsele, `Subtotal`, fiecare reducere aplicată, `TVA 21%`, `TOTAL`. Opțiune nouă în meniu: `6. Bon detaliat`, care îl primește cu `@Qualifier`.
- Pachetul `comenzi/reduceri/`: interfața `Reducere` cu `String nume()` și `double aplica(Comanda comanda, double suma)`. `ReducereMeniu` („Meniu cafea + croissant”): –2 lei dacă există o cafea (Espresso, Cappuccino, Latte) și un Croissant. `ReducereFidelitate` („Fidelitate -5% peste 30”): ×0.95 dacă suma trece de 30.
- `CalculatorPret` primește toate reducerile într-un `List` și le aplică în ordine: întâi meniul, apoi fidelitatea. TVA-ul se calculează după reduceri.
- `ViewCafenea` are un al doilea constructor, fără bonul detaliat (folosește bonul simplu pentru ambele opțiuni). Spring trebuie să-l folosească pe cel complet.

Verificare

Opțiunile `2, 2, 2, 3, 2, 4, 6, 0`:

```
Bon detaliat comanda #1
  Cappuccino                 12.50
  Latte                      13.00
  Croissant                   9.50
  Subtotal                   35.00
  Meniu cafea + croissant    -2.00
  Fidelitate -5% peste 30    -1.65
  TVA 21%                     6.58
  TOTAL                      37.93 lei
```

Cu ordinea reducerilor inversată, aceeași comandă dă `37.81 lei`.

Etapa 5 · Câte obiecte și când se creează

Adnotări noi: `@Scope`, `@Lazy`, `@DependsOn`

Cerințe

- `ComandaRepositoryImpl` are scope `singleton` scris explicit și numără câte instanțe s-au creat (câmp static, metoda statică `getInstante()`).
- `Comanda` devine bean cu scope `prototype`. `ComandaServiceImpl` obține fiecare comandă nouă prin `ObjectProvider`, fără `new`.
- Pachetul `raport/`: `RaportVanzari`, creat doar la prima folosire. Constructorul afișează `[Raport] creat acum`. Metoda `afiseaza()` = lista de la opțiunea 5 + `Instante ComandaRepository: N`.

Opțiunea 5 îl folosește.

- Pachetul `jurnal/`: `JurnalCafenea`, care afișează `[Jurnal] pornit` în constructor. `ProdusRepositoryImpl` afișează `[Produse] repository creat` în constructor și se creează după jurnal, fără să-l primească în constructor.

Verificare

```
[Jurnal] pornit
[Produse] repository creat
```

La pornire nu apare `[Raport]`. La prima opțiune 5 apare `[Raport] creat acum`, iar raportul se termină cu `Instante ComandaRepository: 1`.

Etapa 6 · Ciclul de viață

Adnotări noi: `@PostConstruct`, `@PreDestroy`, `@Bean(initMethod, destroyMethod)`

Cerințe

- `produse.txt` în `resources/`, câte un produs pe linie: `1;Espresso;8.00`, `2;Cappuccino;12.50`, `3;Latte;13.00`, `4;Croissant;9.50`. `ProdusRepositoryImpl` le încarcă după creare și afișează `[Produse] 4 produse incarcate din produse.txt`. Constructorul nu mai adaugă produse.
- La oprirea aplicației, `ComandaRepositoryImpl` scrie `comenzi.txt` (o linie pe comandă: `id;produs,produs`) și afișează `[Comenzi] N comenzi salvate in comenzi.txt`.
- Pachetul `casa` (în afara lui `app`, fără adnotări): `CasaDeMarcat` cu `deschide()` → `[Casa] deschisa`, `inregistreaza(double suma)` și `inchide()` → `[Casa] inchisa. Total zi: X` (2 zecimale). Bean în `AppConfig`; se deschide la pornire și se închide la oprire. `ComandaServiceImpl` înregistrează totalul fiecărei comenzi finalizate.

Verificare

Opțiunile `2, 2, 2, 4, 4, 2, 2, 2, 3, 2, 4, 4, 0` (comenzile #1 = 24.20 și #2 = 37.93). La ieșire:

```
[Casa] inchisa. Total zi: 62.13
[Comenzi] 2 comenzi salvate in comenzi.txt
```

`comenzi.txt`: `1;Cappuccino,Croissant` și `2;Cappuccino,Latte,Croissant`.

Etapa 7 · Condiții și profiluri

Adnotări noi: `@Profile`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean`

Cerințe

- `MeniuTestRepository` implementează `ProdusRepository` cu 2 produse: `1 Espresso 8.00` și `4 Croissant 9.50`. În profilul `dev` se folosește doar el; altfel, doar `ProdusRepositoryImpl`.
- `cafenea.reduceri.activ: true` în `application.yml`. Reducerile există doar când proprietatea e `true`. `CalculatorPret` merge și fără nicio reducere.
- Pachetul `notificari/`: interfața `Notificator` cu `void trimite(String mesaj)` și `ConsolaNotificator` (fără adnotări), care afișează `[Notificare]` + mesaj. Bean în `AppConfig`, creat doar dacă nu există alt `Notificator`. La finalizare: `Comanda #N finalizata`.

Verificare

- `--spring.profiles.active=dev`: opțiunea 1 arată doar Espresso și Croissant.
- `--cafenea.reduceri.activ=false`: bonul detaliat din etapa 4 nu mai are reduceri, `TOTAL 42.35 lei`.
- La finalizare: `[Notificare] Comanda #1 finalizata`. Cu o clasă nouă `SmsNotificator` marcată `@Component`, mesajul `[Notificare]` nu mai apare.

Etapa 8 · Evenimente

Adnotări noi: `@EventListener`

Cerințe

- Pachetul `comenzi/evenimente/`: `record ComandaFinalizataEvent(Comanda comanda)`, publicat de `ComandaServiceImpl` la finalizare, prin `ApplicationEventPublisher`.
- Pachetul `bucatarie/`: `Bucatarie` afișează `[Bucatarie] pregateste #N: produs, produs`.
- Pachetul `statistici/`: `Statistici` numără produsele vândute și afișează `[Statistici] produse vandute azi: N`.
- `JurnalCafenea` afișează `[Jurnal] Cafeneaua e deschisa` la `ApplicationStartedEvent`, înainte de meniu.
- `ComandaServiceImpl` nu știe de `Bucatarie` și nici de `Statistici`.

Verificare

La pornire apare `[Jurnal] Cafeneaua e deschisa`. Finalizarea comenzii Cappuccino + Croissant:

```
[Notificare] Comanda #1 finalizata
[Bucatarie] pregateste #1: Cappuccino, Croissant
[Statistici] produse vandute azi: 2
```

Etapa 9 · Fire și programare în timp

Adnotări noi: `@EnableAsync`, `@Async`, `@EnableScheduling`, `@Scheduled`

Cerințe

- `Bucatarie` rulează pe alt fir: afișează și numele firului, așteaptă 2 s, apoi `[Bucatarie] #N gata`. Meniul nu așteaptă bucătăria.
- `Statistici` numără și comenzile. La fiecare `cafenea.raport.ms` milisecunde (`60000` în `application.yml`) afișează `[Raport] comenzi: N, produse: M`. Contoarele sunt folosite din mai multe fire.
- La opțiunea 0 aplicația se oprește complet, cu mesajele `[Casa]` și `[Comenzi]` din etapa 6.

Verificare

Cu `--cafenea.raport.ms=1500`: după finalizare, meniul apare imediat, apoi:

```
[Bucatarie] pregateste #1: Cappuccino, Croissant (task-1)
[Raport] comenzi: 1, produse: 2
[Bucatarie] #1 gata
```

Ordinea liniilor poate varia de la o rulare la alta.